Linux 进程间通信（IPC）学习验证 — 开发日志

在 Ubuntu 18.04 虚拟机（内核版本 `4.15.0-213-generic`）上，通过 10 组 Linux C 程序学习和验证 11 种常见的进程间通信（IPC）机制的工作原理。每种通信方式在设备上编译运行后，用对应终端命令确认内核对象状态和数据流转，记录真实的运行日志。

所有程序均通过数据搬运（管道/套接字/消息队列）、地址空间共享（mmap/共享内存）或事件通知（信号）实现两个独立进程之间的数据传递。以下按类别逐一说明，每种机制涵盖：概述、程序清单、运行日志、数据流动逻辑图、代码讲解。

目录

- 1. 匿名管道 & 命名管道 FIFO
- 2. UNIX 域套接字
- 3. TCP 套接字
- 4. 文件内存映射
- 5. POSIX 共享内存
- 6. 匿名内存文件 memfd
- 7. System V 共享内存
- 8. POSIX 消息队列
- 9. System V 消息队列
- 10. 信号
- 各类 IPC 的关系与差异

---

1. 匿名管道 & 命名管道 FIFO（Anonymous Pipe & Named Pipe）

概述

管道是最基础的 IPC 机制，数据以**字节流**形式在内核缓冲区中传递，无消息边界。

- **匿名管道** `pipe_demo`：通过 `pipe()` 创建、`fork()` 继承 fd，**仅限有亲缘关系的进程**（父子进程）使用。父写子读，所有写端关闭后读端收到 EOF。
- **命名管道 FIFO**：通过 `mkfifo()` 在文件系统中创建类型为 FIFO 的特殊文件（路径如 `/tmp/ipc_fifo_demo`），**任意进程**按路径 `open()` 即可通信。`open()` 会阻塞直到双方都打开（rendezvous 会合语义）。

涉及的程序

程序	角色	说明
`pipe_demo`	单程序自包含	父进程 fork 子进程，通过匿名管道发送消息
`fifo_writer`	发送方	打开 FIFO 写端，发送消息后关闭（触发读端 EOF）
`fifo_reader`	接收方	创建/验证 FIFO 文件，打开读端，读取消息后 `unlink()` 清理

运行与验证

匿名管道（单终端，程序内部自 fork）：

```
root@ubuntu:/home/tjn/commu-IPC/01-pipe-and-fifo# ./pipe_demo
子进程收到：你好，子进程！这是来自父进程的消息。
root@ubuntu:/home/tjn/commu-IPC/01-pipe-and-fifo#
```

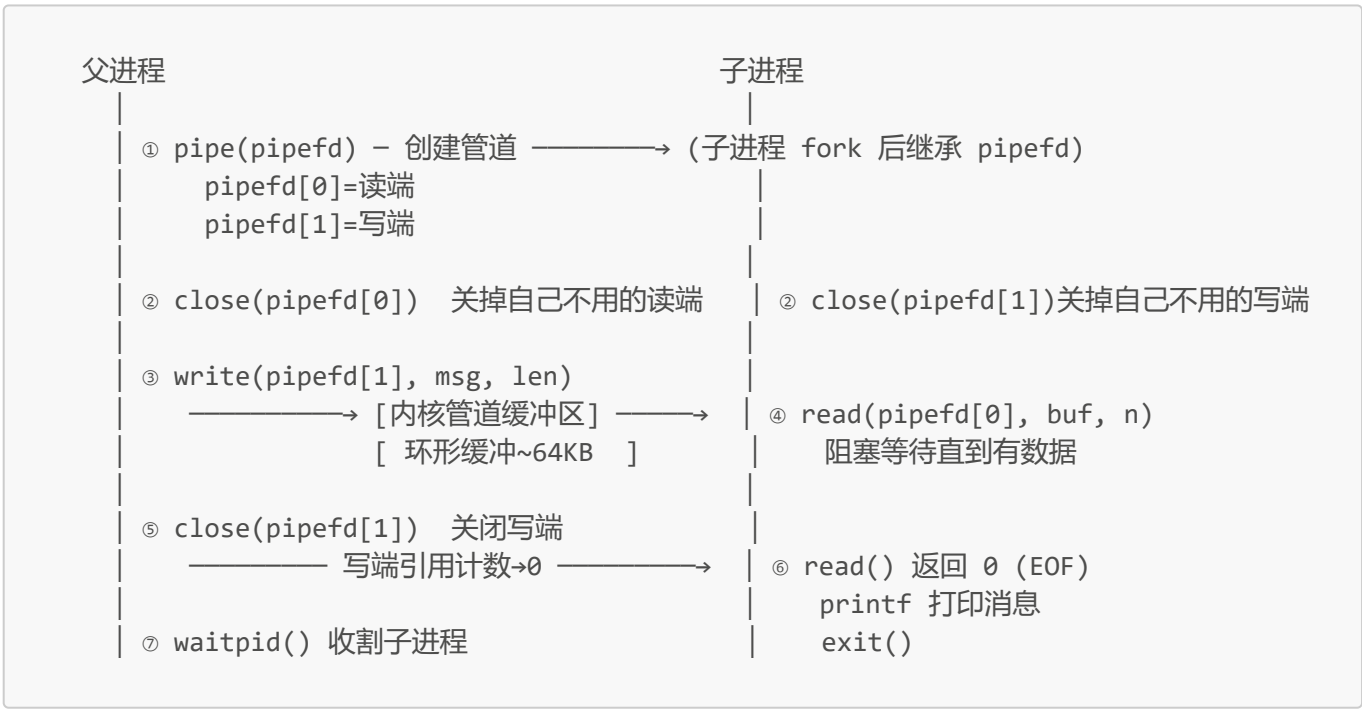
FIFO 命名管道（需两个终端，必须先启动 reader 创建 FIFO 文件）：

```
# 终端 A：reader 创建 FIFO 并阻塞等待 writer 打开
root@ubuntu:/home/tjn/commu-IPC/01-pipe-and-fifo# ./fifo_reader
reader 收到：你好，reader！这是来自独立 writer 的消息。
```

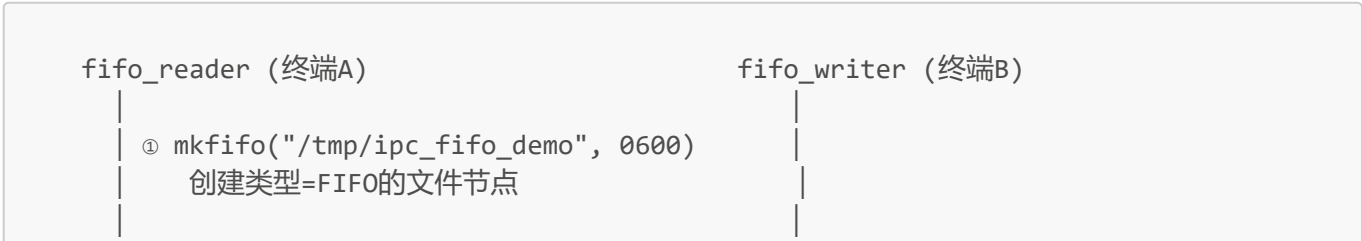
```
# 终端 B：writer 打开 FIFO 写端并发送消息
root@ubuntu:/home/tjn# cd commu-IPC/01-pipe-and-fifo/
root@ubuntu:/home/tjn/commu-IPC/01-pipe-and-fifo# ./fifo_writer
```

数据流动逻辑图

匿名管道（数据全程经内核中转，父子进程不共享任何内存）：



FIFO（双方通过文件系统路径找到同一根管道，open() 会合）：



```
      │ ② open(path, O_RDONLY)  ──阻塞等待写端──→  │
      │                                            │ ③ open(path, O_WRONLY)
      │  ←── rendezvous 会合，双方 open 同时返回 ──┤
      │                                            │
      │ ④ read(fd, buf, n) 阻塞等待数据             │ ⑤ write(fd, msg, len)
      │   ←──────────── [内核管道缓冲区] ────────────┤
      │                                            │ ⑥ close(fd) → 写端关闭
      │ ⑦ read() 返回 0 (EOF)                      │
      │ ⑧ printf 打印                              │
      │ ⑨ unlink(path) 删除 FIFO 文件              │
```

代码讲解

匿名管道 `pipe_demo.c` 关键流程：

```c
// ① 创建管道：pipe() 在内核分配环形缓冲区，返回两个 fd
int pipefd[2];
pipe(pipefd);        // pipefd[0]=读端，pipefd[1]=写端

// ② fork 子进程：fork() 后子进程继承全部 fd
pid_t pid = fork();
if (pid == 0) {
    // === 子进程 ===
    close(pipefd[1]);                          // 子进程关闭自己不用写端
    while ((n = read(pipefd[0], buf, sizeof(buf)-1)) > 0)
        ;                                      // 循环读直到 EOF
    buf[used] = '\0';
    printf("子进程收到：%s\n", buf);
    exit(0);
}
// === 父进程 ===
close(pipefd[0]);                              // 父进程关闭自己不用读端
write_all(pipefd[1], message, strlen(message)); // 写入消息
close(pipefd[1]);                              // 关闭写端 → 子进程收到 EOF
waitpid(pid, &status, 0);                      // 等子进程退出
```

关键点：`write_all()` 循环直到写完所有字节（`write()` 可能短写）；`write()` 返回 0 在管道/socket 上不应发生，视为 `EIO` 错误。

FIFO `fifo_reader.c` 关键流程：

```c
// ① 创建 FIFO 特殊文件
mkfifo("/tmp/ipc_fifo_demo", 0600);   // EEXIST时 lstat 验证是否真的是 FIFO

// ② 打开读端 → 阻塞直到 writer 也打开写端
int fd = open(FIFO_PATH, O_RDONLY);

// ③ 循环读取直到 EOF
while (used < sizeof(buffer)-1) {
```

```
    ssize_t n = read(fd, buffer+used, sizeof(buffer)-1-used);
    if (n == 0) break;                  // writer 关闭写端
    used += n;
}

// ④ 溢出检测：缓冲区满了但还没 EOF？再读1字节试探
if (used == sizeof(buffer)-1) {
    char extra;
    if (read(fd, &extra, 1) > 0)        // 还有数据 → 消息超长
        error("消息超过缓冲区限制");
}

// ⑤ 清理：reader 负责删除 FIFO 文件
unlink(FIFO_PATH);
```

FIFO `fifo_writer.c` 关键流程：

```
// ① 忽略 SIGPIPE → 对端关闭时 write() 返回 EPIPE 而不是杀进程
sigaction(SIGPIPE, &(struct sigaction){.sa_handler=SIG_IGN}, NULL);

// ② 打开写端 → 阻塞直到 reader 也打开读端
int fd = open(FIFO_PATH, O_WRONLY);

// ③ 可靠写入
write_all(fd, message, strlen(message));

// ④ 关闭写端 → reader 收到 EOF
close(fd);
```

关键点：必须先启动 reader。若 writer 先启动 → `open()` 返回 `ENOENT`（FIFO 文件还没创建）。

---

2. UNIX 域套接字（UNIX Domain Socket）

概述

UNIX 域套接字（`AF_UNIX` + `SOCK_STREAM`）提供**全双工字节流**通信。采用客户端/服务器模型：服务器 `bind()` 到文件路径（如 `/tmp/ipc_unix_demo.sock`）、`listen()` 等待连接，客户端 `connect()` 后双方通过 `send()/recv()` 双向传输数据。API 结构和 TCP 套接字完全相同，但不经过网络协议栈，仅限本机通信。

涉及的程序

程序	角色	说明
`unix_server`	服务端	socket→bind→listen→accept，收到请求后回复固定消息
`unix_client`	客户端	socket→connect，发送请求后接收回复并打印

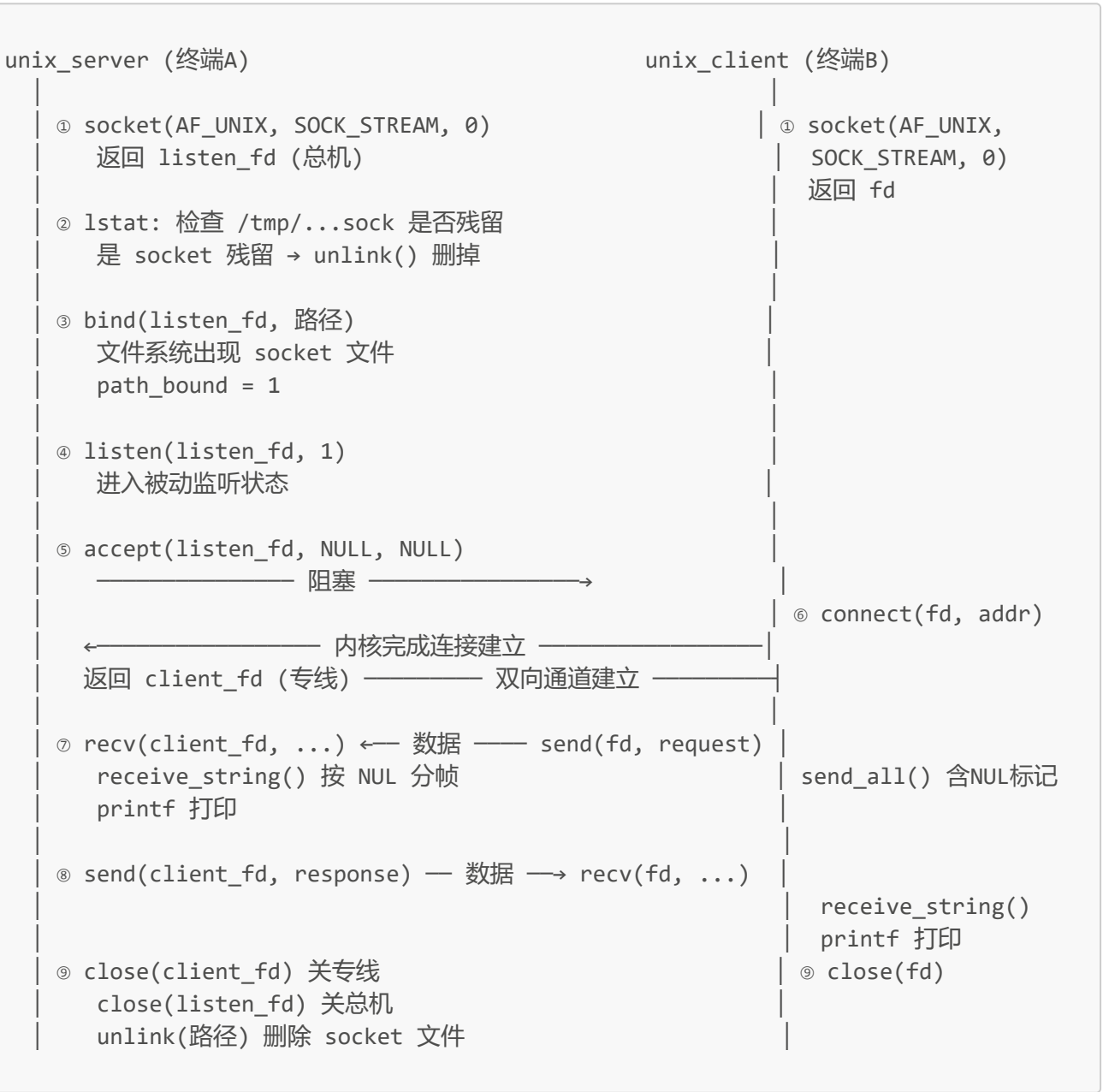
运行与验证

需两个终端，先启动 server，再启动 client：

```
# 终端 A：server 阻塞在 accept() 等待客户端连接
root@ubuntu:/home/tjn/commu-IPC/02-unix-domain-socket# ./unix_server
server 收到：你好，UNIX server!
root@ubuntu:/home/tjn/commu-IPC/02-unix-domain-socket#
```

```
# 终端 B：client 发起连接并发送请求
root@ubuntu:/home/tjn/commu-IPC/02-unix-domain-socket# ./unix_client
client 收到：server 已收到 UNIX 域套接字请求
root@ubuntu:/home/tjn/commu-IPC/02-unix-domain-socket#
```

数据流动逻辑图



代码讲解

`unix_server.c` **关键流程：**

```
// ① 创建 socket + 清理残留 socket 文件
listen_fd = socket(AF_UNIX, SOCK_STREAM, 0);   // AF_UNIX=本机, SOCK_STREAM=流式
// lstat 检查路径是否已存在 socket 残留 → unlink 删除

// ② 绑定路径 → 文件系统出现 /tmp/ipc_unix_demo.sock
struct sockaddr_un addr = {.sun_family = AF_UNIX};
snprintf(addr.sun_path, sizeof(addr.sun_path), "%s", SOCKET_PATH);
bind(listen_fd, (struct sockaddr*)&addr, sizeof(addr));
listen(listen_fd, 1);                          // backlog=1, 只接受一个客户端

// ③ 阻塞等待客户端连接 → 返回新 fd（专线）
client_fd = accept(listen_fd, NULL, NULL);
// accept 返回的 client_fd 和 listen_fd 是不同 fd —— client_fd 用于传数据

// ④ 收→打→回（NUL 分帧协议）
receive_string(client_fd, request, sizeof(request));  // memchr 找 '\0' 确定消息边界
printf("server 收到: %s\n", request);
send_all(client_fd, response, sizeof(response));      // sizeof 包含末尾 NUL
```

`unix_client.c` **关键流程：**

```
// ① 创建 socket（客户端不需要 bind —— 内核自动分配匿名地址）
int fd = socket(AF_UNIX, SOCK_STREAM, 0);

// ② 填充服务器地址并 connect（触发内核与 server 的 accept 配对）
struct sockaddr_un addr = {.sun_family = AF_UNIX};
snprintf(addr.sun_path, sizeof(addr.sun_path), "%s", SOCKET_PATH);
connect(fd, (struct sockaddr*)&addr, sizeof(addr));

// ③ 发→收
send_all(fd, request, sizeof(request));                // 发送 NUL 结尾请求
receive_string(fd, response, sizeof(response));        // 阻塞等待 NUL 结尾回复
printf("client 收到: %s\n", response);
```

NUL 分帧协议：`send_all` 传 `sizeof`（含 `\0`）、`receive_string` 用 `memchr` 找 `\0`——SOCK_STREAM 是字节流无消息边界，必须自定义协议划分消息。

---

3. TCP 套接字（TCP Socket）

概述

TCP 套接字（`AF_INET` + `SOCK_STREAM`）的 API 结构和 UNIX 域套接字完全相同，区别在于寻址方式从"文件路径"变为 **IP + 端口**（`127.0.0.1:8888`）。数据经过完整 TCP/IP 协议栈（三次握手、拥塞控制、重传），本

demo 绑定回环地址 `127.0.0.1` 以保持仅限本机通信。需 `setsockopt(SO_REUSEADDR)` 解决 TCP 特有的 TIME_WAIT 端口占用问题。

涉及的程序

程序	角色	说明
`tcp_server`	服务端	socket→setsockopt→bind(127.0.0.1:8888)→listen→accept
`tcp_client`	客户端	socket→connect(127.0.0.1:8888)，发送请求后接收回复

运行与验证

需两个终端，先启动 server，再启动 client：

```
# 终端 A：server 监听 127.0.0.1:8888
root@ubuntu:/home/tjn/commu-IPC/03-tcp-socket# ./tcp_server
server 收到：你好，TCP server!
root@ubuntu:/home/tjn/commu-IPC/03-tcp-socket#
```

```
# 终端 B：client 向 127.0.0.1:8888 发起连接
root@ubuntu:/home/tjn/commu-IPC/03-tcp-socket# ./tcp_client
client 收到：server 已收到 TCP 请求
root@ubuntu:/home/tjn/commu-IPC/03-tcp-socket#
```

数据流动逻辑图

```
  tcp_server (终端A)                              tcp_client (终端B)
    │                                                 │
    │ ① socket(AF_INET, SOCK_STREAM, 0)               │ ① socket(AF_INET,
    │    AF_INET → 走 IPv4 协议栈                      │   SOCK_STREAM, 0)
    │                                                 │
    │ ② setsockopt(SO_REUSEADDR, 1)                   │
    │    允许 TIME_WAIT 期间快速重启                    │
    │                                                 │
    │ ③ bind(127.0.0.1:8888)                          │
    │    端口 8888 绑定到回环接口                       │
    │    只接受本机连接（不用 0.0.0.0）                  │
    │                                                 │
    │ ④ listen(fd, 1)                                 │
    │                                                 │
    │ ⑤ accept(fd, NULL, NULL)                        │
    │    ──────── 阻塞等待 SYN ────────→               │
    │                                                 │ ⑥ connect(127.0.0.1:8888)
    │   ←──── 三次握手：SYN→SYN+ACK→ACK ──────────────┤
    │   返回 client_fd ──── 连接建立 ─────────────────┤
    │                                                 │
    │ ⑦ recv() ←──── TCP 字节流 ──── send_all() ──────┤
```

```
      |    NUL 分帧解析                                    |
      | ⑧ send_all() ── TCP 字节流 ──→ recv()              |
      |    NUL 分帧解析                                    |
      |                                                    |
      | ⑨ close(client_fd) → 四次挥手 FIN 交换             | ⑨ close(fd) → TIME_WAIT ~60s
      |    close(listen_fd)                                |
```

代码讲解

tcp_server.c 关键流程（和 UNIX socket 同骨架，差异只在地址）：

```
// ① 创建 IPv4 TCP socket + 设置 SO_REUSEADDR
int listen_fd = socket(AF_INET, SOCK_STREAM, 0);  // AF_INET→IPv4，其余同 UNIX
int reuse = 1;
setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse));
// SO_REUSEADDR 解决 TCP 特有的 TIME_WAIT 端口占用问题

// ② 填充 IPv4 地址（和 UNIX socket 不同之处）
struct sockaddr_in addr;
memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_port   = htons(8888);              // 主机序→网络序(大端)，不转会绑错端口
inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr); // "127.0.0.1"→4字节二进制
// 绑定 127.0.0.1 而非 0.0.0.0 → 拒绝非本机连接

bind(listen_fd, (struct sockaddr*)&addr, sizeof(addr));
listen(listen_fd, 1);

// ③ 接受连接 — 背后内核完成三次握手（SYN→SYN+ACK→ACK）
client_fd = accept(listen_fd, NULL, NULL);

// ④ 应用层协议：NUL 分帧（和 UNIX socket 完全相同）
receive_string(client_fd, request, sizeof(request));
send_all(client_fd, response, sizeof(response));
```

tcp_client.c 关键流程：

```
// ① socket + 填充服务器地址
int fd = socket(AF_INET, SOCK_STREAM, 0);
struct sockaddr_in addr = {
    .sin_family = AF_INET,
    .sin_port   = htons(8888)
};
inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);

// ② connect → 触发三次握手，阻塞直到握手完成
connect(fd, (struct sockaddr*)&addr, sizeof(addr));

// ③ 发送+接收（和 UNIX socket 完全相同）
```

```
send_all(fd, request, sizeof(request));
receive_string(fd, response, sizeof(response));
```

关键点：`htons()` 将端口从主机字节序转为网络字节序（大端）；`inet_pton()` 将点分十进制 IP 转为 4 字节二进制；绑定 `127.0.0.1` 而非 `0.0.0.0` 以限制仅本机访问。

---

4. 文件内存映射（File-backed mmap）

概述

`mmap()` 将磁盘文件映射到进程虚拟地址空间——通过**共享映射**（`MAP_SHARED`）使得两进程读写同一物理页，无需 `read()/write()` 系统调用。当 writer 通过指针直接写映射区时，内核负责将脏页同步到文件和其他映射方的页表。这是"零拷贝"通信的开端，但自身不提供同步机制。

涉及的程序

程序	角色	说明
`mmap_writer`	写入方	创建 `/tmp/ipc_mmap_demo.dat` → `ftruncate` → `mmap` → `memcpy` → `msync`
`mmap_reader`	读取方	`open` 已有文件 → `mmap(PROT_READ)` → `memchr` 找 NUL → 打印 → `unlink`

运行与验证

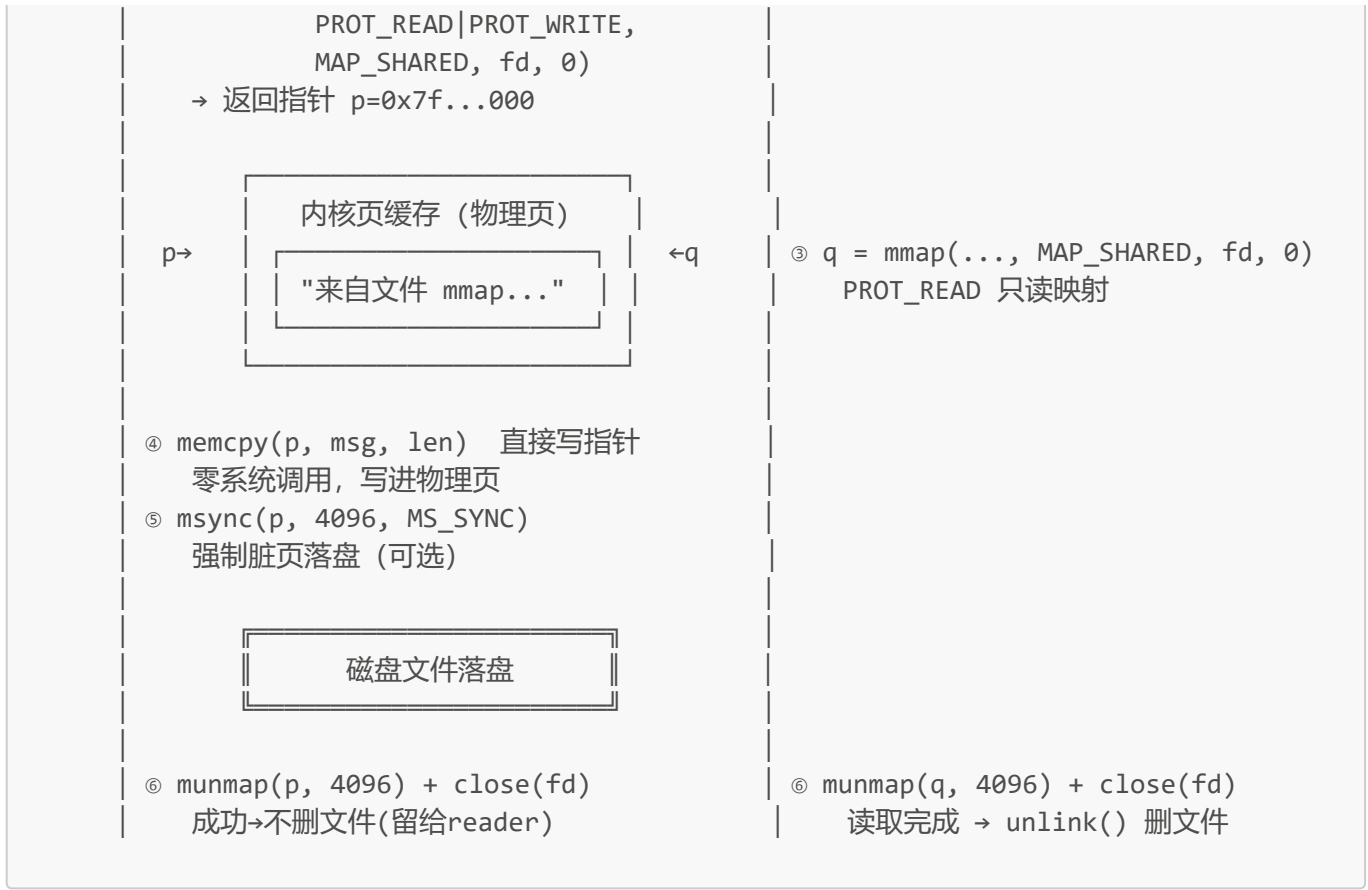
必须先 writer 后 reader（reader 依赖文件已存在）：

```
root@ubuntu:/home/tjn/commu-IPC/04-mmap# ./mmap_writer
已写入 mmap 文件：来自文件 mmap writer 的消息
root@ubuntu:/home/tjn/commu-IPC/04-mmap# cat /tmp/ipc_mmap_demo.dat
来自文件 mmap writer 的消息
root@ubuntu:/home/tjn/commu-IPC/04-mmap# ./mmap_reader
读取到 mmap 文件：来自文件 mmap writer 的消息
root@ubuntu:/home/tjn/commu-IPC/04-mmap#
```

验证关键点：`cat /tmp/ipc_mmap_demo.dat` 能直接看到消息，因为映射写入了磁盘文件。

数据流动逻辑图

```
mmap_writer                                mmap_reader
   │                                           │
   │ ① open("/tmp/ipc_mmap_demo.dat",          │
   │        O_CREAT|O_EXCL|O_RDWR, 0600)       │
   │    → 拿到 fd=3                            │
   │                                           │
   │ ② ftruncate(fd, 4096)                     │
   │    文件撑到 4096 字节（分配物理页）          │
   │                                           │
   │ ③ p = mmap(NULL, 4096,                    │
```



代码讲解

mmap_writer.c 关键流程：

```c
// ① 创建普通文件 → 拿到 fd
int fd = open("/tmp/ipc_mmap_demo.dat", O_CREAT|O_EXCL|O_RDWR|O_NOFOLLOW, 0600);
// EEXIST 时 unlink 残留文件后重建

// ② 设置文件大小（**必须**——不调 ftruncate，mmap 后超出部分一碰就 SIGBUS）
ftruncate(fd, 4096);

// ③ 映射文件到地址空间
char *p = mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
// MAP_SHARED 是关键：写入直接修改共享页，对其他映射同一文件的进程可见
// 如果误用 MAP_PRIVATE：写时复制(CoW) → 只有自己私有页被改 → reader 永远看不到

// ④ 直接写指针——零系统调用
memset(p, 0, 4096);
memcpy(p, message, strlen(message) + 1);    // +1 包含 NUL

// ⑤ 可选：强制脏页落盘（默认内核延迟写回）
msync(p, 4096, MS_SYNC);
```

mmap_reader.c 关键流程：

```
// ① 打开 writer 创建的文件
int fd = open("/tmp/ipc_mmap_demo.dat", O_RDONLY);

// ② fstat 验证文件 ≥ 4096 字节（防御：避免 SIGBUS）
struct stat st;
fstat(fd, &st);
if (st.st_size < 4096) error("文件长度不足");

// ③ mmap 只读映射（PROT_READ）
const char *q = mmap(NULL, 4096, PROT_READ, MAP_SHARED, fd, 0);

// ④ 安全扫描 NUL 终止符
const char *nul = memchr(q, '\0', 4096);   // 绝不用 strlen——越界会 SIGSEGV
printf("%.*s\n", (int)(nul - q), q);

// ⑤ reader 负责清理
unlink("/tmp/ipc_mmap_demo.dat");
```

关键点：`memchr(q, '\0', 4096)` 限制扫描范围——如果 writer 没写 NUL，`strlen(q)` 会越过映射区末尾导致 SIGSEGV。

---

5. POSIX 共享内存（POSIX Shared Memory）

概述

和文件 mmap 同样使用 `mmap(MAP_SHARED)`，但映射的对象不是磁盘文件，而是一个**只存在于 RAM 中的命名内核对象**（底层在 tmpfs `/dev/shm` 中）。使用 `shm_open()` 创建/打开（等价于在 `/dev/shm` 创建匿名文件），`ftruncate()` 设置大小，之后流程和文件 mmap 完全相同。相比文件 mmap，零磁盘 I/O，更快。

涉及的程序

程序	角色	说明
`posix_shm_writer`	写入方	`shm_open(O_CREAT)` 创建 → `ftruncate` → `mmap` → `memcpy`
`posix_shm_reader`	读取方	`shm_open(O_RDONLY)` 打开 → `mmap` → `memchr` 读 → `shm_unlink`

运行与验证

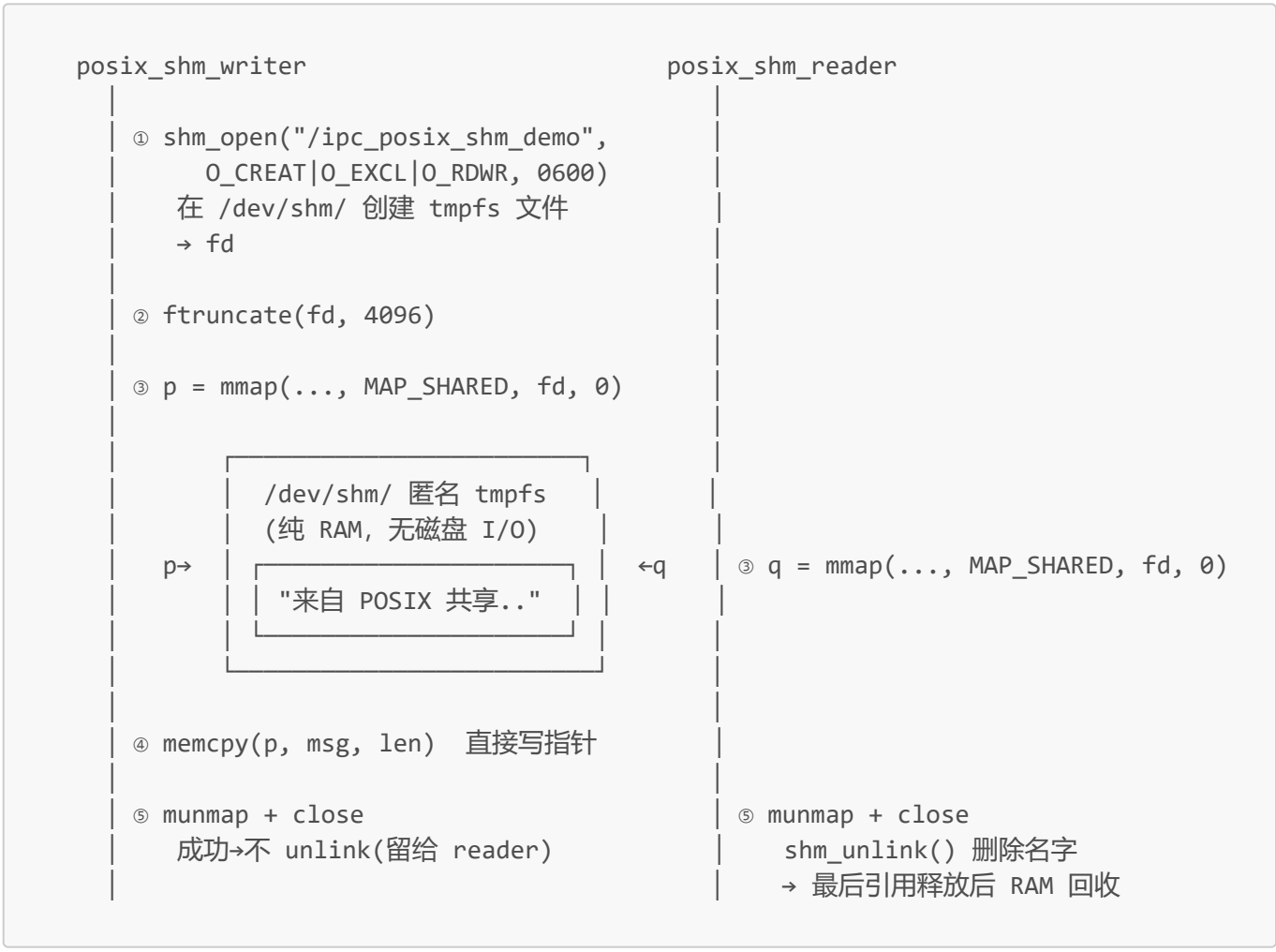
writer 先运行创建对象，reader 后运行读取并销毁：

```
root@ubuntu:/home/tjn/commu-IPC/05-posix-shm# ./posix_shm_writer
已写入 POSIX 共享内存：来自 POSIX 共享内存 writer 的消息
root@ubuntu:/home/tjn/commu-IPC/05-posix-shm# ./posix_shm_reader
读取到 POSIX 共享内存：来自 POSIX 共享内存 writer 的消息
root@ubuntu:/home/tjn/commu-IPC/05-posix-shm# ./posix_shm_writer
已写入 POSIX 共享内存：来自 POSIX 共享内存 writer 的消息
root@ubuntu:/home/tjn/commu-IPC/05-posix-shm# ls -l /dev/shm/
total 4
```

```
-rw------- 1 root root 4096 Aug  6 00:17 ipc_posix_shm_demo
root@ubuntu:/home/tjn/commu-IPC/05-posix-shm# stat /dev/shm/ipc_posix_shm_demo
  File: /dev/shm/ipc_posix_shm_demo
  Size: 4096            Blocks: 8          IO Block: 4096   regular file
Device: 19h/25d Inode: 3           Links: 1
Access: (0600/-rw-------)  Uid: (    0/    root)   Gid: (    0/    root)
Access: 2026-08-06 00:17:59.788333370 -0700
Modify: 2026-08-06 00:17:59.788333370 -0700
Change: 2026-08-06 00:17:59.788333370 -0700
 Birth: -
```

验证关键点：`ls -l /dev/shm/` 看到文件 `ipc_posix_shm_demo` 大小为 4096（=`SHM_SIZE`），`stat` 确认权限 `0600`。

数据流动逻辑图



代码讲解

`posix_shm_writer.c` **关键流程：**

```
// ① 创建 POSIX 共享内存对象（本质：在 /dev/shm 创建 tmpfs 文件）
int fd = shm_open("/ipc_posix_shm_demo", O_CREAT|O_EXCL|O_RDWR, 0600);
// 和 open() 语法一致，但创建的是内核里的匿名对象，名字在 /dev/shm 可见
// EEXIST 时直接 shm_unlink 删掉重建（不需要像 FIFO 那样 lstat 验证类型）
```

```
// ② ftruncate（新对象长度=0，必须设置大小）
ftruncate(fd, 4096);

// ③ mmap —— 和文件 mmap 完全相同
char *p = mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);

// ④ 指针直写
memset(p, 0, 4096);
memcpy(p, message, strlen(message) + 1);

// ⑤ 成功时保留对象给 reader（失败时才 shm_unlink）
```

`posix_shm_reader.c` **关键流程：**

```
// ① 打开已有对象（无 O_CREAT —— sender 必须已创建）
int fd = shm_open("/ipc_posix_shm_demo", O_RDONLY, 0);

// ② fstat 验证大小 ≥ 4096（防御 SIGBUS）
struct stat st;
fstat(fd, &st);

// ③ mmap 只读映射
const char *q = mmap(NULL, 4096, PROT_READ, MAP_SHARED, fd, 0);

// ④ 安全读：memchr 限界扫描
const char *nul = memchr(q, '\0', 4096);

// ⑤ reader 负责删除
shm_unlink("/ipc_posix_shm_demo");  // 删名字，对象等引用归零后销毁
```

与文件 mmap 的对比：仅将 `open()` 换成 `shm_open()`、`unlink()` 换成 `shm_unlink()`。核心映射逻辑 mmap 完全相同。

---

6. 匿名内存文件 memfd（memfd_create）

概述

`memfd_create()` 创建一个**没有文件系统名字的匿名 RAM 文件**，仅通过 fd 引用。和 POSIX 共享内存一样需要通过 `fork()` 继承 fd 才能在进程间共享（无法被无亲缘关系的进程通过路径打开）。最大的优势是生命周期极简：**关闭最后一个 fd 自动回收内存，无需任何 unlink/IPC_RMID 操作。**

涉及的程序

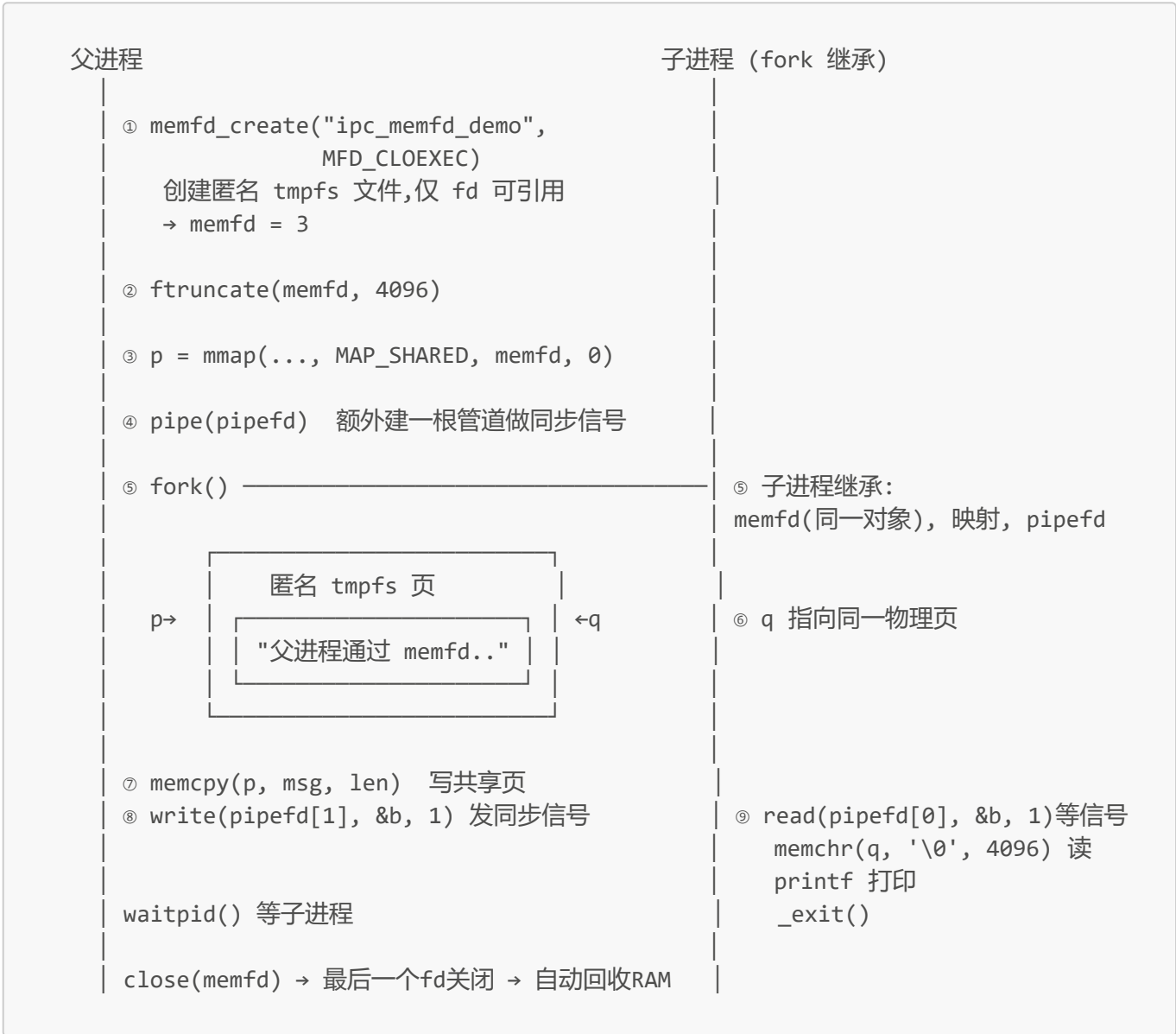
程序	角色	说明
`memfd_demo`	单程序自包含	父进程 `memfd_create`→`ftruncate`→`mmap`→`fork`；子进程从共享映射读取

运行与验证

```
root@ubuntu:/home/tjn/commu-IPC/06-memfd# ./memfd_demo
子进程读取 memfd: 父进程通过 memfd 写入的消息
父进程已写入并等待子进程读取完成
root@ubuntu:/home/tjn/commu-IPC/06-memfd#
```

验证关键点：memfd 无文件系统路径，只能通过 `/proc/<pid>/fd/` 看到 `memfd:ipc_memfd_demo (deleted)`。程序退出后自动消失，零残留。

数据流动逻辑图



代码讲解

```
// ① memfd_create —— 创建无名的 tmpfs 文件
int memfd = memfd_create("ipc_memfd_demo", MFD_CLOEXEC);
// 名字仅调试可见（/proc/pid/fd 里显示 memfd:ipc_memfd_demo (deleted)）
// MFD_CLOEXEC: exec() 时不继承 —— 安全
```

```
// ② ftruncate + mmap（和其他共享内存一样）
ftruncate(memfd, 4096);
char *p = mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_SHARED, memfd, 0);

// ③ 额外建一根匿名管道做同步信号
int pipefd[2];
pipe(pipefd);  // [0]=读，[1]=写

// ④ fork → 子进程继承 memfd、映射、pipefd
pid_t pid = fork();
if (pid == 0) {
    // === 子进程 ===
    close(pipefd[1]);
    char dummy;
    read(pipefd[0], &dummy, 1);  // 阻塞等父进程写完
    // 现在可以安全读共享页了
    printf("子进程读取 memfd: %s\n", (const char *)q);
    _exit(0);
}
// === 父进程 ===
close(pipefd[0]);
memcpy(p, message, strlen(message) + 1);  // 写共享页
write(pipefd[1], "", 1);                   // 发同步信号给子进程
close(pipefd[1]);
waitpid(pid, NULL, 0);
// close(memfd) → 最后一个 fd 关闭 → RAM 自动回收，无需 unlink/IPC_RMID
```

关键点：memfd 用额外的管道解决共享内存缺乏同步的问题——父写完 → 管道写 1 字节 → 子 `read()` 返回才敢读映射。这是典型的"用管道弥补共享内存无同步"模式。

---

7. System V 共享内存（System V Shared Memory）

概述

System V 共享内存是 POSIX SHM 的**前身**（1980 年代老 API），功能相同但接口风格完全不同：

- 用 `ftok(文件路径, proj_id)` 推导整数 **key**（而非名字字符串）定位对象
- 用 `shmat()/shmdt()` 挂载/摘除（而非 `mmap/munmap`）
- 用 `shmctl(IPC_RMID)` 立即销毁（而非 `shm_unlink` 惰性删除）

内核对象在 System V IPC 命名空间中，可通过 `ipcs -m` 直接查看。

涉及的程序

程序	角色	说明
`sysv_shm_writer`	写入方	`ftok→shmget(IPC_CREAT)→shmat→memcpy→shmdt`

程序	角色	说明
sysv_shm_reader	读取方	shmget→shmctl(IPC_STAT) 验大小 →shmat(SHM_RDONLY)→memchr→shmdt→IPC_RMID

运行与验证

```
root@ubuntu:/home/tjn/commu-IPC/07-system-v-shm# ./sysv_shm_writer
已写入 System V 共享内存：来自 System V 共享内存 writer 的消息
root@ubuntu:/home/tjn/commu-IPC/07-system-v-shm# ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x5301002e 65536      root       600        4096       0

root@ubuntu:/home/tjn/commu-IPC/07-system-v-shm# ./sysv_shm_reader
读取到 System V 共享内存：来自 System V 共享内存 writer 的消息
root@ubuntu:/home/tjn/commu-IPC/07-system-v-shm#
```

验证关键点：`ipcs -m` 列出所有 System V 共享内存段——`bytes=4096` 验证了 `shmget` 分配的大小正确。残留段可手动清理：`ipcrm -m <shmid>`。

数据流动逻辑图



```
      │ ⑤ shmdt(p)  摘挂载（段还在）                  │ ⑥ shmdt(q)
      │    成功→不删段(留给 reader)                   │    shmctl(id, IPC_RMID) 立即销毁
```

代码讲解

`sysv_shm_writer.c` **关键流程：**

```c
// ① ftok 推导 key
key_t key = ftok("/tmp/ipc_sysv_shm_demo.key", 'S');
// 内部：取文件的 st_dev(文件系统) + st_ino(inode) 低8位 + proj_id低8位 → 哈希成int
// 如果文件被删除重建(inode变了)，双方算出不同 key → 找不到同一个段！

// ② shmget 创建段（返回整数 id 而非 fd）
int id = shmget(key, 4096, IPC_CREAT|IPC_EXCL|0600);
// IPC_EXCL：存在则失败 → 触发清理残留段后重试

// ③ shmat 挂载（POSIX 用 mmap，SysV 用 shmat）
char *p = shmat(id, NULL, 0);
// 返回指针，之后和 mmap 一样直接读写

// ④ memcpy 写（和 POSIX SHM 完全相同）
memcpy(p, message, strlen(message) + 1);

// ⑤ shmdt 摘挂载（POSIX 用 munmap）
shmdt(p);
// 成功时保留段给 reader
```

`sysv_shm_reader.c` **关键流程：**

```c
// ① ftok → shmget（不用 IPC_CREAT，段必须已存在）
key_t key = ftok("/tmp/ipc_sysv_shm_demo.key", 'S');
int id = shmget(key, 4096, S_IRUSR|S_IWUSR);

// ② shmctl(IPC_STAT) 验证段大小（防御）
struct shmid_ds info;
shmctl(id, IPC_STAT, &info);
if (info.shm_segsz < 4096) error("段大小不足");

// ③ shmat 只读挂载
const char *q = shmat(id, NULL, SHM_RDONLY);

// ④ memchr 扫描 → 打印
// ⑤ shmdt + shmctl(IPC_RMID) 立即销毁
shmdt(q);
shmctl(id, IPC_RMID, NULL);  // IPC_RMID 是立即销毁，不等引用归零！
```

关键点：`shmctl(IPC_RMID)` 是**立即销毁**（和 POSIX 的 `shm_unlink` 惰性删除相反），调用瞬间段就被标记删除，任何后续的 shmat 都失败。

8. POSIX 消息队列（POSIX Message Queue）

概述

POSIX 消息队列将通信模型从"字节流"改为 **"离散消息 + 优先级"**。每条 `mq_send()` 都是一条**独立的完整消息**，内核保留边界；多条消息按**优先级从高到低**出队（同优先级 FIFO）。队列创建时需固化 `mq_maxmsg`（容量）和 `mq_msgsize`（单条上限），之后不可改。需链接 `-lrt`。

涉及的程序

程序	角色	说明
`posix_mq_sender`	发送方	`mq_open(O_CREAT)` 创建队列→`mq_send(优先级5)` 发送一条消息
`posix_mq_receiver`	接收方	`mq_open(O_RDONLY)` 打开→`mq_receive` 阻塞等待→打印优先级和内容→`mq_unlink`

运行与验证

必须先 sender 后 receiver（队列需要被创建）：

```
root@ubuntu:/home/tjn/commu-IPC/08-posix-message-queue# ./posix_mq_sender "hello
mq"
已发送: hello mq (优先级 5)
root@ubuntu:/home/tjn/commu-IPC/08-posix-message-queue# ./posix_mq_receiver
已接收: hello mq (优先级 5)
```

验证关键点：若需查看队列对象，挂载 mqueue 文件系统后 `ls /dev/mqueue`。

数据流动逻辑图

```
  posix_mq_sender                              posix_mq_receiver
    │                                              │
    │ ① mq_open("/ipc_posix_mq_demo",              │
    │       O_CREAT|O_EXCL|O_RDWR|O_NONBLOCK,      │
    │       0600, &attr)                           │
    │     attr = {maxmsg=10, msgsize=256}          │
    │     固化 → 之后不可改                          │
    │     EEXIST→打开已有队列→mq_getattr 验属性       │
    │     不匹配→mq_unlink 删掉重建                  │
    │     匹配→mq_receive 循环排空残留消息           │
    │     (O_NONBLOCK → EAGAIN=队列空了)            │
    │                                              │
    │ ② mq_send(q, "hello mq", 8, 5)               │
    │     ─── 整条入队 ──→                          │
    │         ┌──────────────────────┐             │
```

```
      │        │   POSIX 消息队列（内核）   │        │
      │        │  maxmsg=10, msgsize=256  │        │
      │        │  ┌──────────────────┐    │        │
      │        │  │ prio=5: "hello mq"│    │        │ ③ mq_open(O_RDONLY)
      │        │  └──────────────────┘    │        │    mq_receive 阻塞等待
      │        │  优先级高者先出队          │        │    ← 取出 prio=5 的消息
      │        └──────────────────────────┘        │
      │                                            │
      │                                            │
      │ ③ mq_close(q)                              │ ④ mq_close(q)
      │    成功→不 unlink(留队列给 receiver)        │    mq_unlink() 删队列名字
      │                                            │    → 最后一个 close 后内核销毁
```

代码讲解

posix_mq_sender.c 关键流程：

```c
// ① 定义队列属性（创建时固化，之后不可改）
struct mq_attr attr = {
    .mq_maxmsg  = 10,    // 最多存 10 条消息
    .mq_msgsize = 256,   // 单条消息最多 256 字节
    .mq_flags   = 0
};

// ② 创建队列（O_EXCL → 存在则失败）
mqd_t q = mq_open("/ipc_posix_mq_demo", O_CREAT|O_EXCL|O_RDWR|O_NONBLOCK, 0600,
&attr);
if (q == -1 && errno == EEXIST) {
    // 队列已存在 → 打开并检查属性
    q = mq_open("/ipc_posix_mq_demo", O_RDWR|O_NONBLOCK);
    struct mq_attr actual;
    mq_getattr(q, &actual);
    if (actual.mq_maxmsg != 10 || actual.mq_msgsize != 256) {
        // 属性不匹配 → mq_unlink 删掉重建（因为创建后属性不可改）
        recreate_queue(&q);
    }
    // 排空残留消息（O_NONBLOCK 模式，EAGAIN=空了）
    char stale[256];
    while (mq_receive(q, stale, sizeof(stale), NULL) >= 0);
}

// ③ 发送消息
mq_send(q, message, strlen(message), 5);
// 参数：（队列，内容，长度，优先级）
// 优先级 5：数字越大越先出队；同优先级按 FIFO
// 不会"短写"—要么整条成功，要么失败

// ④ sender 不 unlink —— 队列留给 receiver
mq_close(q);
```

posix_mq_receiver.c 关键流程：

```
// ① 打开已有队列（无 O_CREAT）
mqd_t q = mq_open("/ipc_posix_mq_demo", O_RDONLY);

// ② mq_getattr 验证 mq_msgsize ≤ 接收缓冲区
struct mq_attr attr;
mq_getattr(q, &attr);
if (attr.mq_msgsize > MESSAGE_SIZE) error("缓冲区太小");

// ③ 阻塞接收一条消息
ssize_t n = mq_receive(q, buffer, MESSAGE_SIZE, &priority);
// mq_receive 返回这条消息的实际字节数
// priority 输出这条消息的优先级
// 注意：cap 必须 ≥ mq_msgsize，否则返回 EMSGSIZE

// ④ receiver 负责删除
mq_unlink("/ipc_posix_mq_demo");  // 删名字，等所有 close 后销毁
```

与管道/套接字的本质区别：一次 `mq_receive` = 恰好一条完整消息，不用像管道那样循环 `receive_string` 找 `'\0'` 边界。

---

9. System V 消息队列（System V Message Queue）

概述

System V 消息队列是 POSIX 消息队列的老版本。每条消息携带一个 `long mtype`（正类型标签），接收端可按类型过滤（`msgrcv(..., msgtyp)`），而不是 POSIX 的优先级排序。同样用 `ftok()` 推导整数 key 来定位对象，用 `ipcs -q` 可查看队列状态。

涉及的程序

程序	角色	说明
`sysv_mq_sender`	发送方	`ftok`→`msgget(IPC_CREAT)`→`msgsnd(type=1)`
`sysv_mq_receiver`	接收方	`msgget`→`msgrcv(type=1)` 按类型过滤→`IPC_RMID` 销毁

运行与验证

先 sender 创建队列并发送，再 receiver 接收并销毁：

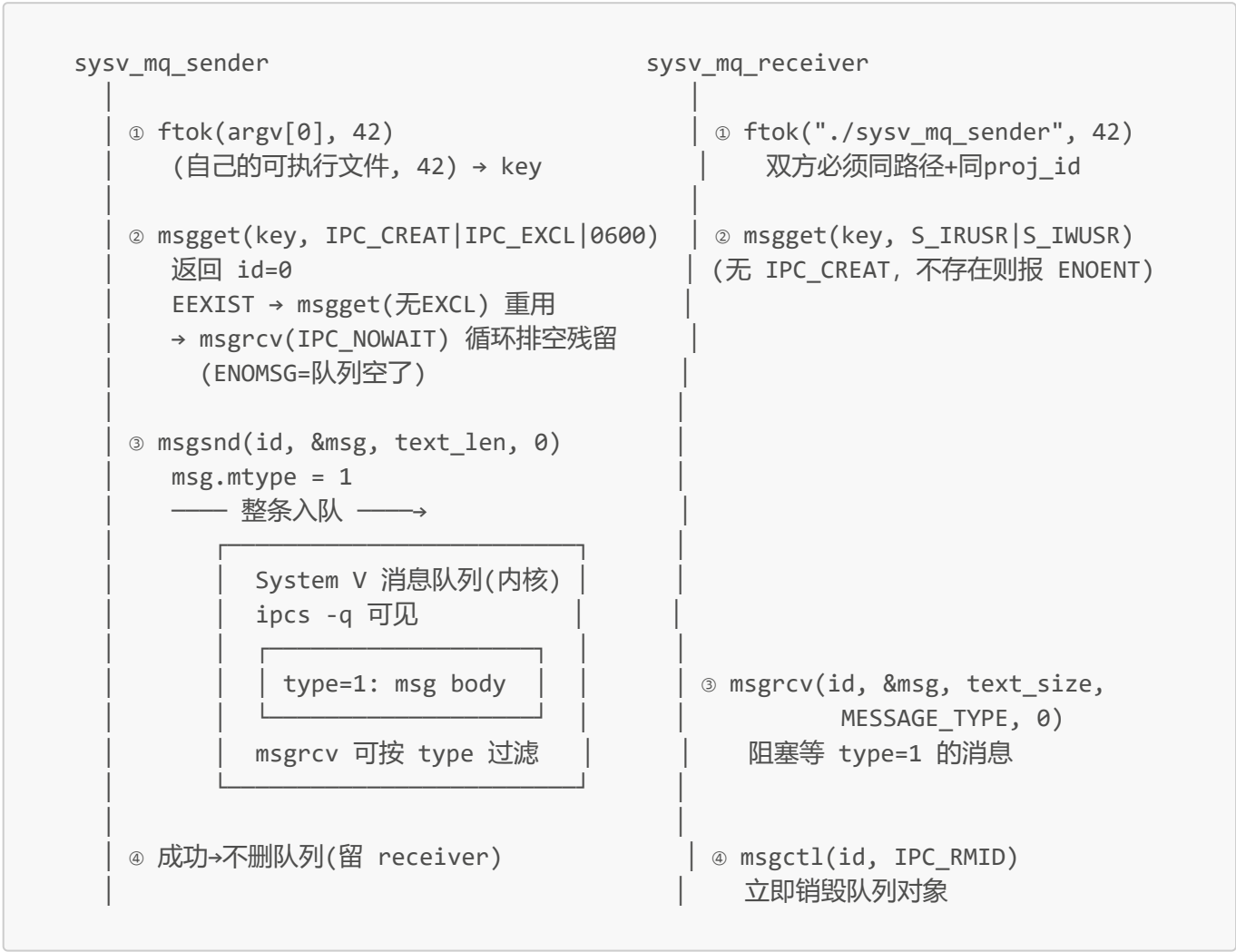
```
root@ubuntu:/home/tjn/commu-IPC/09-system-v-message-queue# ./sysv_mq_sender
已发送 type=1：来自 System V 消息队列的问候
root@ubuntu:/home/tjn/commu-IPC/09-system-v-message-queue# ipcs -q

------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x4d01002e 0          root       600        38           1
```

```
root@ubuntu:/home/tjn/commu-IPC/09-system-v-message-queue# ./sysv_mq_receiver
已接收 type=1：来自 System V 消息队列的问候
```

验证关键点：`ipcs -q` 显示队列中 `messages=1`、`used-bytes=38`——证明消息确实在内核队列中。残留可手动清理：`ipcrm -Q <key>`。

数据流动逻辑图



代码讲解

消息结构体（必须首位为 `long mtype`）：

```c
struct message {
    long mtype;           // 必须是第一个成员，且为正数
    char text[255];       // 应用层负载
};
// msgsnd/msgrcv 的 size 参数只算 text 部分（sizeof(text)），不包含 mtype
```

`sysv_mq_sender.c` 关键流程：

```c
// ① ftok 从自己的可执行文件推导 key（和 SysV SHM 一样）
key_t key = ftok(argv[0], 42);

// ② 创建队列（IPC_CREAT|IPC_EXCL）
int id = msgget(key, IPC_CREAT|IPC_EXCL|S_IRUSR|S_IWUSR);
if (id == -1 && errno == EEXIST) {
    // 残留队列 → 重用并排空
    id = msgget(key, IPC_CREAT|S_IRUSR|S_IWUSR);
    // 非阻塞排空残留消息（IPC_NOWAIT → ENOMSG=空了）
    struct message stale;
    while (msgrcv(id, &stale, sizeof(stale.text), 0, IPC_NOWAIT) >= 0);
}

// ③ 发送 mtype=1 的消息
struct message msg = {.mtype = 1};   // 必须设置正数
memcpy(msg.text, payload, strlen(payload) + 1);
msgsnd(id, &msg, strlen(payload) + 1, 0);
// 参数：(队列id, 消息指针, body长度, flags) —— flags=0 表示阻塞
```

`sysv_mq_receiver.c` **关键流程：**

```c
// ① ftok + msgget（无 IPC_CREAT）
key_t key = ftok("./sysv_mq_sender", 42);
int id = msgget(key, S_IRUSR|S_IWUSR);

// ② msgrcv 按 mtype 过滤接收
struct message msg;
ssize_t n = msgrcv(id, &msg, sizeof(msg.text), 1, 0);
// 参数：(id, 消息指针, body最大长度, msgtyp, flags)
// msgtyp=1：只收 mtype==1 的消息
// msgtyp=0：收任意类型（先进先出）
// msgtyp<0：收 mtype≤|msgtyp| 的最小者
// flags=0：阻塞等待；IPC_NOWAIT：无匹配消息时返回 ENOMSG

msg.text[n] = '\0';
printf("收到消息（类型 %ld，%zd 字节）：%s\n", msg.mtype, n, msg.text);

// ③ IPC_RMID 立即销毁
msgctl(id, IPC_RMID, NULL);
```

关键点：`msgrcv` 的 `msgtyp` 过滤是 SysV MQ 独有的——POSIX MQ 没有类型过滤、只有优先级排序。

---

10. 信号（Signal）

概述

信号是一种**事件通知**机制，负载仅 4/8 字节。实时信号（`SIGRTMIN..SIGRTMAX`）相比标准信号的关键优势是**可排队**：标准信号阻塞期间多次发送只投递一次，实时信号每次都会记录（受 `RLIMIT_SIGPENDING` 限制）。

以下程序演示：receiver 注册 `SIGRTMIN+1` 的 SA_SIGINFO 三参数处理函数 + `sigsuspend` 原子等待，sender 通过 `sigqueue()` 向目标 PID 发送信号并携带整数负载。

涉及的程序

程序	角色	说明
`signal_receiver`	接收方	`sigaction(SA_SIGINFO)` 注册处理函数→打印 PID→`sigsuspend` 阻塞等待
`signal_sender`	发送方	`sigqueue(<PID>, SIGRTMIN+1, value)` 发送信号+整数

运行与验证

必须先 receiver 后 sender（sender 需要 receiver 的 PID）：

```
# 终端 A：receiver 打印 PID 后阻塞等待信号
root@ubuntu:/home/tjn/commu-IPC/10-signal# ./signal_receiver
接收端 PID：34666，等待信号 SIGRTMIN+1
收到信号，携带整数：2026
```

```
# 终端 B：sender 向 PID 34666 发送 SIGRTMIN+1，负载整数 2026
root@ubuntu:/home/tjn/commu-IPC/10-signal# ./signal_sender 34666 2026
已向 PID 34666 发送 SIGRTMIN+1，整数为 2026
```

数据流动逻辑图

```
  signal_receiver (终端A)                    signal_sender (终端B)
    │                                            │
    │ ① sigemptyset + sigaddset(SIGRTMIN+1)      │
    │    sigprocmask(SIG_BLOCK, &set, &old)      │
    │    先阻塞信号 → 防止注册后立即被投递          │
    │                                            │
    │ ② sigaction(SIGRTMIN+1, &act, NULL)        │
    │    act.sa_sigaction = handle_signal        │
    │    act.sa_flags = SA_SIGINFO  ← 关键!      │
    │    注册三参数处理函数                        │
    │    (处理函数里只做 sig_atomic_t 赋值)        │
    │                                            │
    │ ③ printf PID (34666) 给用户看               │
    │    fflush(stdout) 强制刷新                  │
    │                                            │
    │ ④ sigsuspend(&wait_mask)                   │
    │    原子操作：解除屏蔽 + 睡眠等待             │
    │    ──────────── 阻塞 ──────────→           │
    │                                            │ ⑤ sigqueue(34666,
```

```
      |                                                |       SIGRTMIN+1,
      |                                                |       {sival_int=2026})
      |   ←──── 内核投递：handler 被异步调用 ────────|
      |   handle_signal(sig, info, ctx)                |
      |     received_value = info->si_value            |
      |                      .sival_int (=2026)        |
      |     signal_received = 1                        |
      |                                                |
      | ⑥ sigsuspend 返回(EINTR)                       |
      |    while(!signal_received) 退出循环            |
      |    printf("收到信号，携带整数：2026")          |
```

代码讲解

signal_receiver.c 关键流程：

```c
// ① 先阻塞 SIGRTMIN+1（防止处理函数注册窗口期收到信号）
sigset_t blocked, old_mask;
sigemptyset(&blocked);
sigaddset(&blocked, SIGRTMIN + 1);
sigprocmask(SIG_BLOCK, &blocked, &old_mask);

// ② 注册 SA_SIGINFO 三参数处理函数
struct sigaction act = {0};
act.sa_sigaction = handle_signal;
act.sa_flags = SA_SIGINFO;   // ★ 必须有此标志才能收到 si_value 负载
sigaction(SIGRTMIN + 1, &act, NULL);

// ③ 打印 PID 给 sender
printf("接收端 PID: %ld, 等待信号 SIGRTMIN+1\n", (long)getpid());
fflush(stdout);

// ④ 原子解除阻塞 + 睡眠等待（sigsuspend 核心）
sigset_t wait_mask = old_mask;
sigdelset(&wait_mask, SIGRTMIN + 1);
while (!signal_received) {
    sigsuspend(&wait_mask);   // 原子：换掩码(解除阻塞) + 睡眠
    // 返回 -1 + EINTR 说明处理函数执行完毕
}

// 信号处理函数（仅做最少工作—信号安全函数）：
static void handle_signal(int sig, siginfo_t *info, void *ctx) {
    (void)sig; (void)ctx;
    if (info != NULL)
        received_value = info->si_value.sival_int;  // 读出 sender 附带的整数
    signal_received = 1;  // volatile sig_atomic_t, 和主线程的 while 同步
}
```

signal_sender.c 关键流程：

```
// ① 解析 PID（严格校验）
pid_t pid = (pid_t)parsed_pid;

// ② 选信号号（SIGRTMIN+1 而非 +0——+0 被 glibc pthread 内部占用）
int sig = SIGRTMIN + 1;

// ③ 构造负载 + 发送
union sigval value;
value.sival_int = (int)parsed_value;  // 负载：4字节整数
sigqueue(pid, sig, value);
// sigqueue 对比 kill(pid, sig): sigqueue 可携带 union sigval 负载
// kill 只能发信号，不能带数据
```

关键点：`sigsuspend()` 是**原子操作**——如果拆成 `sigprocmask(unblock) + pause()`，信号可能在两者之间到达 → 丢失。`sigsuspend` 把"解除阻塞 + 等待"合并为一次系统调用，消除这个竞态。

各类 IPC 的关系与差异

匿名管道是最原始的 IPC——`pipe()` 在内核分配一块缓冲区，靠 `fork()` 继承 fd 让父子进程各拿一端，数据从写端拷进内核、再从内核拷到读端，进程退出管道自动消失。它只能单向传、只能亲缘进程用。**FIFO** 把同样的管道模型搬到了文件系统上——`mkfifo()` 创建一个路径，任意两个进程按路径 `open()` 就能通信，不再需要 fork 亲缘关系。但数据流方式没变：依然是单向字节流，`open()` 多了会合阻塞，文件需要 `unlink()` 显式清理。

UNIX 域套接字在 FIFO 的基础上做了两步升级：把单向变成全双工（双方都能同时收发），把简单的打开/读写模型换成客户端/服务器模型（bind/listen/accept/connect）。它仍然用文件路径寻址，仍然只在本机通信。**TCP 套接字**和 UNIX 域套接字的 API 结构完全一样，只是把路径地址换成 IP+端口，数据经过完整的网络协议栈——这意味着它可以跨主机通信（本 demo 绑定 127.0.0.1 限制在本机）。多出的 `htons()` 处理字节序、`SO_REUSEADDR` 解决 TIME_WAIT 问题，都是网络地址带来的额外代价。

以上四种都是"字节流"模型——数据没有边界，发端写 20 字节、收端可能分两次读到，必须自己设计分帧协议（本系列统一用 NUL 字节标记消息结束）。

文件 mmap 换了一种完全不同的思路——不再把数据搬来搬去，而是把同一个文件页同时映射进两个进程的地址空间，双方通过指针直接读写同一块物理内存，零系统调用。代价是内核不再帮你同步——写方写完读方未必知道，需要外部协调。**POSIX 共享内存**把文件 mmap 里的"磁盘文件"换成纯 RAM 里的命名对象（在 `/dev/shm`），其余流程完全一样，少了磁盘 I/O。**memfd** 再进一步去掉名字，对象只能靠 `fork()` 继承 fd，换来的是 `close()` 自动回收——不需要 unlink。**System V 共享内存**是 POSIX 共享内存的前身，用 `ftok()` 推导的整数 key 定位、用 `shmat()` 映射、用 `IPC_RMID` 立即销毁——功能相同，API 完全不同。

POSIX 消息队列回到了搬运型模型，但解决了字节流的核心痛点——`mq_send()` 和 `mq_receive()` 以整条消息为单位，内核天然保留边界，不用自己分帧。还多了优先级排序：数字越大越先出队。**System V 消息队列**是它的老版，用 `mtype` 标签过滤代替优先级排序，同样用 key 定位和 `IPC_RMID` 销毁。

信号是唯一的"事件通知"机制——不传数据块，只向目标 PID 投递一个信号加上最多 4 字节整数。实时信号相比标准信号的关键改进是可排队不丢，配合 `sigsuspend` 原子等待消除了竞态。